Tuş Vuruşu Dinamiği ile Parola Doğrulama

BİL462 Biyometrik Tanıma · Proje A5 · Okuma kılavuzu: ne yaptık, nasıl yaptık, neden böyle yaptık

Bahadır İhsan Herdem (221101034) · Ömer Efe Öztürk (221101059) · 17 Ekim 2026 · Durum: veri analizi, dedektörler ve değerlendirme tamamlandı; tasarım dokümanı, demo ve ikinci geliştirme sırada

> **Bu belge ne işe yarar?** Hocaya vereceğimiz bir belge değil, bizim için bir okuma kılavuzu. Projedeki her kararın arkasındaki gerekçeyi anlatır; böylece sunumda “neden SVM?”, “neden oturuma göre ayırdınız?”, “bu sayı ne demek?” sorularına kendi cümlelerinizle cevap verebilirsiniz. Hocanın kuralı açık: sunumda kodun ve sonucun her parçasını açıklayabilmemiz gerekiyor. Aşağıdaki yeşil kutular (*Neden böyle?*) tam olarak bunun içindir.

1. Proje bir bakışta

Soru (hocanın A5 tanımı): Aynı parolayı yazan kullanıcıları yazma ritimlerinden ayırabilir miyiz ve günler geçtikçe bu ayrım ne kadar bozulur?

Ana sonuç: Oturum içi ile oturumlar arası hata oranı arasındaki fark. Hocanın deyişiyle “aynı oturumda ölçülen başarım yanıltıcı derecede iyidir, çünkü kişinin o günkü ritmi sabittir.”



Şu ana kadar yapılanlar

- Veri analizi ve 31 özelliğin incelenmesi (bölüm 3)
- Dört dedektörün kendi kodumuzla yazılması: Manhattan, Scaled Manhattan, Mahalanobis, One-class SVM (bölüm 5)
- Makalenin sonuçlarının yeniden üretilmesi (doğrulama testi): **başarılı** (bölüm 6.1)
- Oturum içi, oturumlar arası, zamanla bozulma, skor birleştirme ve eşik seçimi deneyleri (bölüm 6)

Kod klasörü: `~/Desktop/462Project/proje/`. Her sayıyı `run_all.py` ile tek komutta yeniden üretebiliyoruz (hocanın istediği “sonuçları yeniden üreten betik”).

2. Temel kavramlar (kısa)

Doğrulama (1:1)

Kullanıcı “ben Bahadır’ım” der ve parolasını yazar. Sistem tek bir soruyu yanıtlar: *bu yazış gerçekten Bahadır’a ait mi?* “Bu kişi kim?” sorusu (1:N tanımlama) değildir. Demomuz da bu yüzden bir doğrulama demosudur.

İki hata türü

Terim	Anlamı	Günlük dilde
FMR	Yabancının yanlışlıkla kabul edilme oranı	Güvenlik hatası: yabancı içeri girdi
FNMR	Gerçek kullanıcının yanlışlıkla reddedilme oranı	Rahatsızlık hatası: sen girdin, sistem almadı
EER	FMR = FNMR olacak şekilde eşik seçilince ortak değer	Tek sayıyla özet; düşük iyi
DET eğrisi	Eşik değiştikçe FMR ile FNMR'nin ödünleşimi	Bütün çalışma noktaları; sol alta yakın iyi

> **Neden tek bir "doğruluk yüzdesi" vermiyoruz?** Çünkü iki hata birbirine bağlıdır: eşiği gevşetirsek gerçek kullanıcı rahat girer ama yabancı da girer; sıkarsak tersi olur. "%90 doğruluk" hangi eşikte olduğunu söylemez. Hocanın tekrar tekrar vurguladığı şey bu: sayının ne anlama geldiğini savunabilmek.

Anomali (tek sınıf) dedektörü

Her kullanıcı için yalnızca **o kullanıcının** örnekleriyle bir model eğitilir. Model "bu kişinin normal yazışı nasıl?" sorusunu öğrenir. Yeni bir giriş geldiğinde modelin normalinden ne kadar uzak olduğunu bir **anomali skoru** olarak verir (büyük skor = daha çok yabancı). Eşik üstündeyse reddedilir.

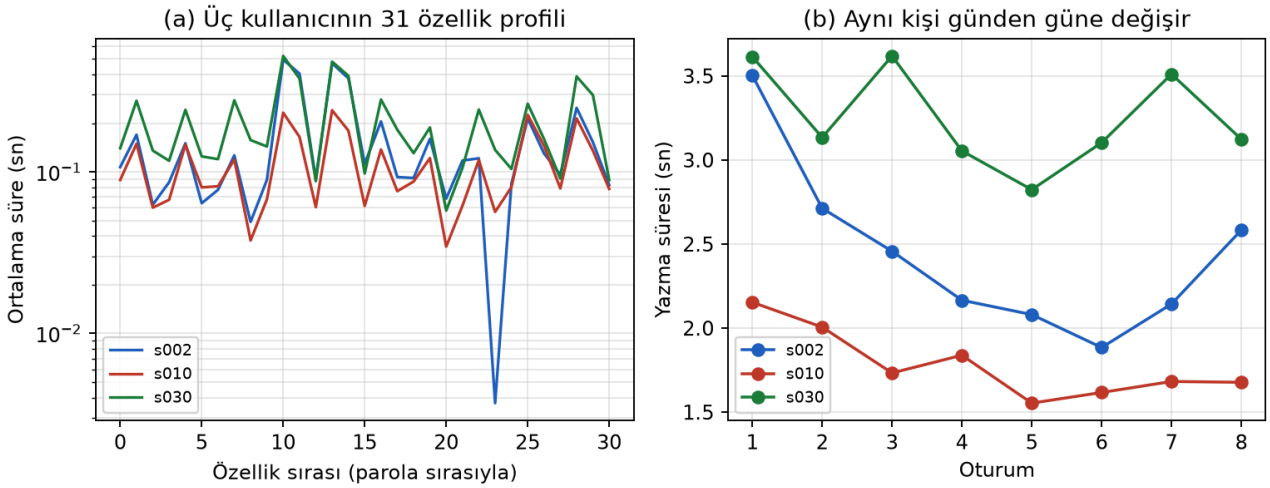
> **Neden her kullanıcıya ayrı model, neden 51 kişilik tek sınıflandırıcı değil?** Gerçek bir sistemde yeni kullanıcı kayıt olduğunda elimizde sadece onun örnekleri vardır, başkalarının verisi yoktur ve her yeni kayıtta herkesi yeniden eğitmek istemeyiz. Makale de (Killourhy & Maxion, 2009) aynı senaryoyu kullanır. Bonus: her kullanıcının kendi modeli olduğundan hocanın "kişi-ayrık bölme" kuralı burada geçerli değildir; onun yerine zamana göre bölme kuralı geçerlidir.

3. Veri: CMU Keystroke Dynamics Benchmark

İçeriği: 20,400 giriş = 51 kişi × 8 oturum × 50 tekrar (kişi başına 400). Herkes **aynı** parolayı yazar: `.tie5Roanl` + Enter (Shift ile R ve 5 gibi büyük/özel karakterler içerir). Oturumlar arasında en az bir gün bırakılmıştır, böylece günden güne değişim ölçülebilir.

Özellikler (31 adet): 11 tuş-basılı-kalma süresi (**H**, bir tuşun basılı kaldığı süre), 10 tuştan-tuşa süre (**DD**, bir tuşa basıştan sonrakine basışa), 10 bırakma-basma süresi (**UD**, bir tuşu bırakıştan sonrakine basışa). 11 tuş (10 karakter + Enter) olduğu için 11 + 10 + 10 = 31.

> **Neden bu veri?** Hocanın A5 için belirttiği tek veri bu; herkese açık, küçük bir metin dosyası. Daha önemlisi referans makalenin de aynı verisi: yani sonuçlarımızı yayımlanmış sayılarla doğrudan karşılaştırabiliriz. Lisans/kullanım koşullarını tasarım dokümanı yazarken CMU sayfasından ayrıca teyit edeceğiz.



Şekil 1. (a) Üç kullanıcının ortalama zamanlama profili: herkesin “parmak izi” farklı. (b) Aynı kişi günden güne değişiyor (oturum ortalamaları). Not: (a)’daki çukur, değeri sıfıra yakın olan bir UD özelliğidir; UD negatif olabildiği için log ölçekte böyle görünür.

Veri kalitesi (9 Kasım kontrol noktası sunumu için not)

Gözlem	Değer
Eksik değer	0
Ortalama H / DD / UD	90 ms / 249 ms / 159 ms
Parolayı yazma süresi (medyan, %99, en uzun)	2.3 sn, 6.7 sn, 36 sn
Kişi başına medyan yazma süresi aralığı	1.4 – 4.9 sn (kişiler arası büyük fark = iyi ayırt edici sinyal)
Medyanın 3 katından uzun girişler	%0,88 (duraksama/hata olabilir; şimdilik silmedik)
Negatif UD değeri	22,118 (hata değil: sonraki tuşa, öncekini bırakmadan basıldı; hızlı yazanlarda normal)

Dürüstlük notu: Şimdiye kadar hiçbir örneği reddetmedik. Hocanın checkpoint sunumunda "preprocessing'de reddedilen örnekler" istiyor; elimizde reddedilen yok, bunu açıkça söyleyeceğiz. İstersek aykırı girişleri temizleyen bir varyant (ör. "Manhattan filtered") ekleyip etkisini ölçebiliriz.

4. Deney tasarımı: neden böyle bölüyoruz?

4.1 Zamana göre bölme ve sızıntı önleme

Hocanın kuralı: eğitim **önceki** oturumlardan, test **sonraki** oturumlardan; test verisiyle ne eğitim ne ayar yapılır; ayar/eğitimde kullanılan kişiler testte yabancı olarak da görünmez (kendi kayıt örnekleri hariç).

Neden bu kadar katı? Rastgele bölseydik aynı oturumdan örnekler hem eğitime hem teste girerdi. Kişinin o günkü ritmi sabit olduğundan model "bu oturumu" ezberlerdi ve sonuç gerçekçi olmayacak kadar iyi çıkardı. Bizim projenin konusu tam olarak bu iyimserliği ölçmek.

4.2 Ayar grubu ve değerlendirme grubu

51 kişiyi rastgele (sabit tohum 0) ikiye ayırdık: **10 ayar kişisi** (s002, s003, s005, s012, s017, s019, s028, s033, s041, s048) ve **41 değerlendirme kişisi**. SVM'nin hiperparametreleri yalnızca ayar kişilerinin ilk 4 oturumuyla seçildi. Bütün sonuçlar değerlendirme kişileri üzerinde hesaplanır.

Neden? Hiperparametre seçmek de bir tür eğitimdir. Aynı kişilerle hem seçip hem raporlarsak sonuç iyimser olur. Böylece hocanın "ayarda kullanılan kişiler testte yer almamalı" kuralını sağlıyoruz. Bedeli: 51 yerine 41 kişi ve yabancı sayısı 40'a düşüyor.

4.3 Üç ayrı koşul

Koşul	Eğitim	Test (gerçek kullanıcı)	Test (yabancı)	Ne için?
C1 oturum içi	Her oturumun ilk 25 girişi	Aynı oturumun son 25 girişi	Diğer 40 kişinin aynı oturumdaki son 25 girişi	İyimser taban
C2 oturumlar arası	Oturum 1–4 (200 giriş)	Oturum 5–8 (200 giriş)	Diğer 40 kişinin oturum 5–8 girişleri (8000)	Gerçekçi senaryo
C3 makale protokolü	İlk 200 giriş	Kalan 200 giriş	Her yabancının ilk 5 girişi	Makaleyle karşılaştırma ve kod doğrulaması
Zamanla bozulma	Oturum 1'in ilk 25 girişi	Oturum t'nin son 25 girişi (t=1…8)	Aynı oturumdaki diğer kişiler	Temiz karşılaştırma

Önemli tuzak (kendi tasarımımızda yakaladık): C1 ile C2'yi doğrudan kıyaslamak adil değil, çünkü C1'de 25, C2'de 200 örnekle eğitiyoruz; eğitim verisi farkı oturum farkıyla karışıyor. Bu yüzden eğitim ve test boyutunu sabit tutan **zamanla bozulma** deneyini ekledik: aynı 25 girişle eğitim, ama test günden güne uzaklaşıyor. Temiz kanıt bu.

4.4 Neden bu metrikler ve bu eşik kuralı?

- **Kullanıcı başına EER:** Her kullanıcı için ayrı hesaplanır, sonra ortalaması alınır (makale de böyle). Bazı kullanıcılar çok tutarlı, bazıları çok değişken yazar; ortalama bunu gizler, bu yüzden dağılımı da gösteriyoruz.
- **EER'in eşiği:** EER'de eşik test skorlarına bakılarak seçilir (makalenin de yaptığı gibi). Bu bir *çalışma noktası değil, karşılaştırma ölçüsüdür* ve iyimser kalır. Gerçekçi olanı için ayrıca "eşiği sadece eğitim döneminden seç" deneyi yaptık (bölüm 6.5).
- **Güven aralığı:** 41 kullanıcıyı yerine koyarak 2000 kez yeniden örnekleyip ortalama EER'in %95 aralığını bulduk. Hocanın istediği "denekleri yeniden örnekleyen bootstrap" budur. Neden kişileri örnekliyoruz, girişleri değil? Aynı kişinin girişleri birbirine benzer; bağımsız değiller. Bağımsız birim kişidir.
- **Düşük FMR'de rapor:** Hata oranını ancak karşılaştırma sayımızın destekleyeceği düzeyde veririz. Kullanıcı başına 8000 yabancı girişle FMR %0,1 düzeyi (8 olay) sınırda destekleniyor; daha düşüğünü raporlamıyoruz.

5. Dedektörler: ne yapıyorlar, neden bunlar?

Hocanın şartı: üç uzaklık tabanlı dedektör + bir öğrenen model; en az bir aşama bizim kendi kodumuz. Hepsini `keystroke/detectors.py` içinde numpy ile kendimiz yazdık (SVM için `scikit-learn` çekirdeği kullandık ve bunu raporda beyan edeceğiz).

Dedektör	Nasıl çalışır?	Güçlü / zayıf yanı
Manhattan	Eğitim girişlerinin ortalama vektörünü alır. Test girişi için 31 özelliğin her birinin ortalamadan farkının mutlak değerlerini toplar.	Çok basit. Ama bütün özellikleri eşit sayar; oysa tuştan tuşa süreler (DD) basılı kalma sürelerinden (H) çok daha oynaktır, böylece oynak özellikler skoru domine eder.
Scaled Manhattan	Aynı, ama her özelliğin farkını o özelliğin eğitimdeki ortalama mutlak sapmasına böler.	Oynak özelliklerin etkisini azaltır. Makalenin 14 dedektör içinde birincisi.
Mahalanobis	Ortalamadan uzaklığı özelliklerin *birbirleriyle ilişkisini* de hesaba katarak ölçer (kovaryans matrisinin tersi).	Korelasyonu görür; ama 31×31 kovaryansı az örnekle kestirmek zordur (C1'de 25 örnek var, bu yüzden küçük bir düzenlileştirme ekledik).
One-class SVM	Eğitim girişlerini çekirdek (RBF) ile yüksek boyutlu bir uzaya taşır ve onları orijinden ayıran bir sınır öğrenir. Sınırın içi "bu kişinin normali".	Doğrusal olmayan sınır öğrenebilir; hiperparametreleri var (ν ve γ).

> **Makalenin en önemli dersi ve bizim sonuçlarımızla uyumu:** Makale, en iyi dedektörlerin ortak noktasının özellikleri **ölçeklemek** olduğunu söylüyor (Euclidean → Mahalanobis, Manhattan → Scaled Manhattan gibi). Bizim sonuçlarımızda da Manhattan'dan Scaled Manhattan'a geçmek hatayı yaklaşık %16'dan %10'a indiriyor.

5.1 Öğrenen model olarak neden One-class SVM?

Seçenekler arasında şu karşılaştırmayı yaptık (makalenin Tablo 2'deki sayıları, EER):

Aday	Makaledeki EER	Karar ve gerekçe
One-class SVM	%10,2 (en iyi 4'te)	**Seçildi.** Yalnızca kullanıcının kendi verisiyle eğitilir, makalede güçlü, doğrudan karşılaştırılabilir.
Neural Network (auto-assoc)	%16,1	Makalede SVM'den belirgin kötü; 500 devirlik eğitim ve ek ayar gerektiriyor.
Neural Network (standard)	%82,8	Makalede yalnızca "1" çıktısı verecek şekilde eğitilmiş (hiç negatif örnek yok); işe yaramadı.
Fuzzy Logic	%22,1	Elle tasarlanmış kümeler, zayıf.
k-Means	%37,2	Zayıf.
Paylaşımlı sinir ağı gömüsü (Siamese)	Makalede yok	Daha modern; **ikinci geliştirme olarak** ekleyeceğiz (bölüm 8).

SVM'yi seçme gerekçelerimiz (dürüst sürüm):

1. Hocanın "bir öğrenen model" şartını sağlıyor.
2. Referans makalede ölçülmüş: kendi sayımızı yayımlanmış %10,2 ile kıyaslayabiliyoruz.
3. Tek sınıf: kayıt anında yalnızca kullanıcının verisi gerekir; bu, demoda da doğrudan geçerli (kayıt oturumlarının verisi ile birkaç saniyede eğitilir).
4. Küçük veriyle çalışır (25–200 örnek) ve skorları *yumuşak*, yani birden fazla girişin skoru birleştirilince çok kazanır (bölüm 6.4).
5. scikit-learn'de hazır, ama hiperparametre seçimi, ölçekleme ve skor üretimi bizim kodumuz.

SVM "en doğru" model mi? Hayır, ve bunu raporda söyleyeceğiz. Makale protokolünde (C3) Scaled Manhattan, SVM'den hafif iyi çıktı; farklar güven aralıkları içinde. Tezimiz "SVM kazandı" olmayacak, "her dedektör zamanla bozuluyor, ama farklı hızda ve SVM'nin skor birleştirmeye en yatkın olduğu görüldü" olacak. Hocanın dediği gibi: olumsuz ama doğru ölçülmüş sonuç değerlidir.

5.2 SVM nasıl eğitildi?

1. **Ölçekleme:** Kullanıcının eğitim girişlerinin her özelliği için ortalama çıkarılıp standart sapmaya bölünür (z-skor). *Neden?* RBF çekirdeği mesafeye duyarlı; saniye cinsinden 1,2 olan DD ile 0,09 olan H birlikte kullanılırsa büyük sayılar hâkim olur.
2. **Model:** `OneClassSVM(kernel="rbf", nu=ν, gamma=γ)`; skor olarak karar fonksiyonunun eksi işaretlisi kullanılır (büyük = yabancı).
3. **Hiperparametre seçimi:** $\nu \in \{0{,}01; 0{,}05; 0{,}1; 0{,}2; 0{,}5\} \times \gamma \in \{0{,}003; 0{,}01; 0{,}03; 0{,}1; 0{,}3\}$ ızgarası; 10 ayar kişisinde oturum 1–3 ile eğitim, oturum 4 ile doğrulama, hedef ortalama EER. Seçilen: **$\nu = 0.05$, $\gamma = 0.03$** (ayar EER'i ≈ %10,5).

Neden ν ve γ? ν, eğitim örneklerinin en fazla ne kadarının sınır dışında kalabileceğini belirler (küçük ν = geniş, hoşgörülü sınır). γ, sınırın ne kadar "dalgalı" olacağını belirler (büyük γ = ezberlemeye eğilim). Izgara sonuçları ν = 0,01–0,2 aralığında γ = 0,03'te hep ≈ %10,5–10,6 verdi: seçim hassas bir ayara bağlı değil, bu iyi bir işaret (şans eseri bir noktaya oturmadık).

6. Sonuçlar

6.1 Doğrulama testi: makaleyi yeniden üretebiliyor muyuz?

C3 protokolüyle (makalenin protokolü) kendi kodumuzun sonuçlarını makaledeki sayılarla karşılaştırdık:

Dedektör	Makale (51 kişi)	Bizim (51 kişi)	Bizim (41 kişi)
Manhattan	%15,3	%15,3 [12,8–18,0]	%16,0 [13,2–18,9]
Scaled Manhattan	%9,6	%9,6 [7,6–11,6]	%10,2 [8,1–12,6]
Mahalanobis	%11,0	%11,1 [9,2–12,9]	%11,1 [9,3–13,2]
One-class SVM	%10,2	— (ayar kişileri dahil olmasın diye hesaplanmadı)	%11,6 [9,5–14,1]

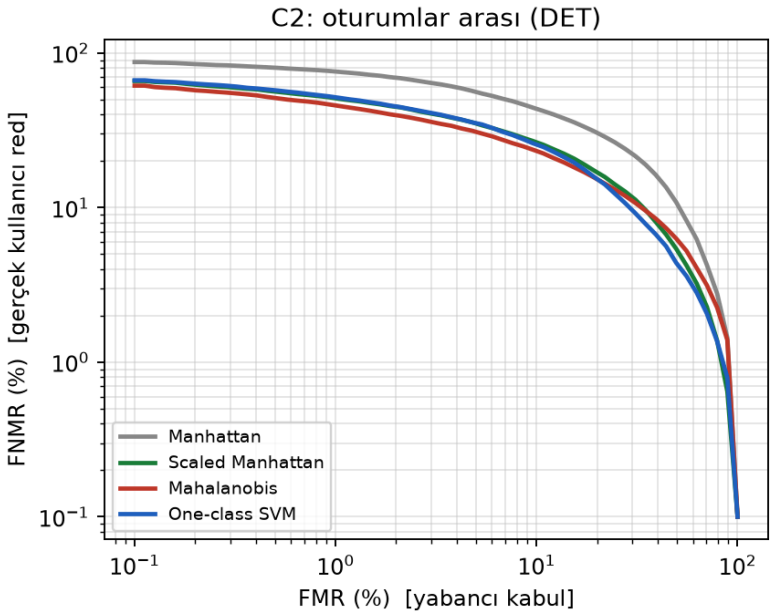
> **Ne gördük?** Manhattan ve Scaled Manhattan makaleyle birebir (%15,3 ve %9,6), Mahalanobis neredeyse aynı (%11,1 / %11,0). Yani veri hattımız, özellik çıkarımımız, EER hesabımız ve dedektör kodlarımız doğru çalışıyor. SVM'de makaledeki uygulama ayrıntıları farklı olduğundan birebir aynı sayı beklemiyoruz (bizim %11,6, makalenin %10,2'si); fark güven aralığımız içinde. Bu test, "kod hatalı mı?" sorusunu baştan kapatır.

6.2 Ana karşılaştırma: oturum içi ve oturumlar arası

Dedektör	C1 oturum içi EER	C2 oturumlar arası EER	C2 – C1 farkı (puan)	C2: FNMR @ FMR=%1	C2: FNMR @ FMR=%0,1
Manhattan	%21,6 [19,2–24,0]	%22,3 [18,5–26,1]	+0,7 [-2,1–3,6]	%76	%87
Scaled Manhattan	%12,3 [10,6–14,1]	%15,1 [12,0–18,4]	+2,7 [0,9–4,8]	%51	%66
Mahalanobis	%18,3 [16,4–20,1]	%15,3 [12,9–17,8]	-2,9 [-4,2–-1,7]	%46	%62
One-class SVM	%14,7 [13,1–16,5]	%14,3 [11,5–17,2]	-0,5 [-2,4–1,6]	%52	%67

EER değerlerindeki köşeli parantez: kullanıcılar üzerinden %95 bootstrap güven aralığı. FNMR @ FMR: kullanıcı başına eşik (sonradan), kişi ortalaması. n = 41 kullanıcı.

- Doğrudan okuma: Scaled Manhattan için C2, C1'den **yaklaşık 2,7 puan kötü** ve güven aralığı sıfırı içermiyor. Mahalanobis'te tersi görülüyor çünkü C2'de 200 örnekle eğitiliyor (kovaryansı kestirmek kolaylaşıyor); bu, eğitim boyutu karışıklığının canlı kanıtı. Bu yüzden asıl cevabı bölüm 6.3'te arıyoruz.
- Bütün dedektörler için FMR %1'de FNMR %46–76 arasında. Yani sıkı güvenlik ayarında gerçek kullanıcıların çoğu reddedilir. Makale de aynı şeyi söylüyor: tuş vuruşu dinamiği tek başına erişim denetimi için yeterli değil; parolaya *ikinci katman* olarak anlamlı.

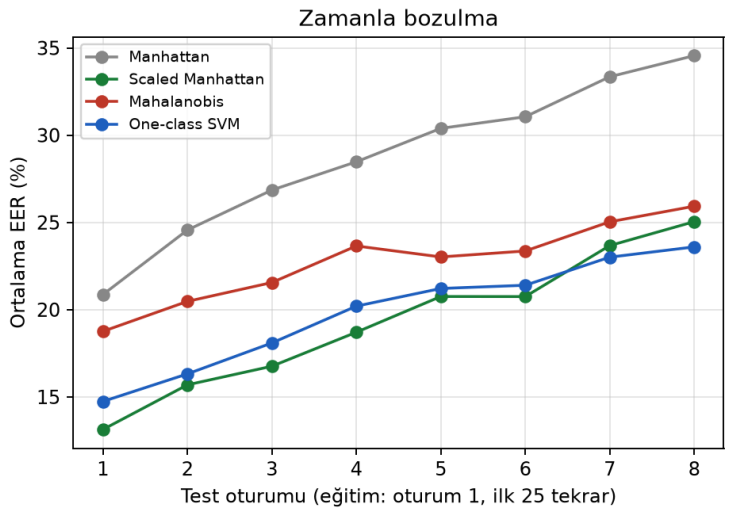


Şekil 2. C2 için DET eğrileri (kullanıcı başına eğrilerin ortalaması; FMR ekseni %0,1'den başlıyor çünkü altı için yeterli yabancı denememiz yok). Sol alta yakın daha iyi. Manhattan belirgin biçimde kötü; diğer üçü birbirine çok yakın.

6.3 Temiz kanıt: zamanla bozulma

Eğitim: oturum 1'in ilk 25 girişi. Test: oturum t'nin son 25 girişi. Eğitim ve test boyutu hep aynı, değişen tek şey **eğitimden ne kadar sonra** test edildiği.

Dedektör	Oturum 1	Oturum 2	Oturum 3	Oturum 4	Oturum 5	Oturum 6	Oturum 7	Oturum 8
Manhattan	%20,9	%24,6	%26,9	%28,5	%30,4	%31,1	%33,3	%34,6
Scaled Manhattan	%13,1	%15,7	%16,8	%18,7	%20,8	%20,8	%23,7	%25,0
Mahalanobis	%18,8	%20,5	%21,6	%23,7	%23,0	%23,4	%25,0	%25,9
One-class SVM	%14,7	%16,3	%18,1	%20,2	%21,2	%21,4	%23,0	%23,6



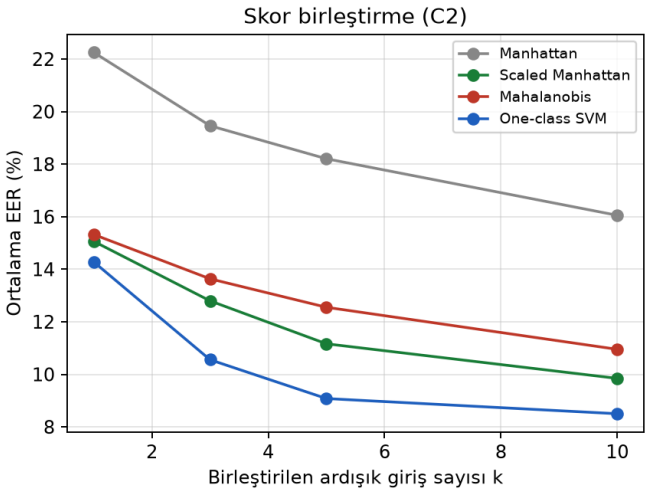
Şekil 3. Ortalama EER, oturum numarasına göre. Oturum 1 aynı oturum (taban), 2–8 daha sonraki günler.

> **Ne gördük?** Dört dedektörde de hata günler geçtikçe yaklaşık monoton artıyor. SVM: %14,7 → %23,6, Scaled Manhattan: %13,1 → %25,0, Manhattan: %20,9 → %34,6. Yani projenin sorusuna cevabımız: **aynı oturumdaki başarım iyimser; sistem yalnızca ilk günün verisiyle eğitilirse hata göreli olarak yaklaşık %40–90 artıyor (dedektöre göre 1,4–1,9 kat).** Dikkat: bu eğrinin güven aralığını henüz hesaplamadık ve oturum 8'de bozulmanın bir kısmı "alışma etkisi"nden gelebilir (insanlar parolayı yazdıkça hızlanıyor, Şekil 1b). Bunları tasarım dokümanında sınırlılık olarak yazacağız.

6.4 Skor birleştirme (isteğe bağlı kısım, yaptık)

Bir girişe karar vermek yerine ardışık **k** girişin anomali skorlarının ortalamasını alıp ona karar veriyoruz (C2 verisi, oturum içinde örtüşmeyen pencereler).

Dedektör	k=1	k=3	k=5	k=10
Manhattan	%22,3	%19,5	%18,2	%16,1
Scaled Manhattan	%15,1	%12,8	%11,2	%9,8
Mahalanobis	%15,3	%13,6	%12,6	%11,0
One-class SVM	%14,3	%10,6	%9,1	%8,5



Neden işe yarıyor? Tek bir giriş gürültülüdür (o an dikkatsiz yazmış olabilirsin). Birkaç girişin ortalaması gürültüyü azaltır; gerçek kullanıcının skorları düşük kalır, yabancınınkiler yüksek kalır. **En çok kazanan SVM:** %14,3 → %8,5 (k=10). Bedel: kullanıcı 10 kez yazmak zorunda kalır, bu yüzden gerçek hayatta "oturumda birkaç deneme" senaryosuna uyar; tek seferlik girişe değil.

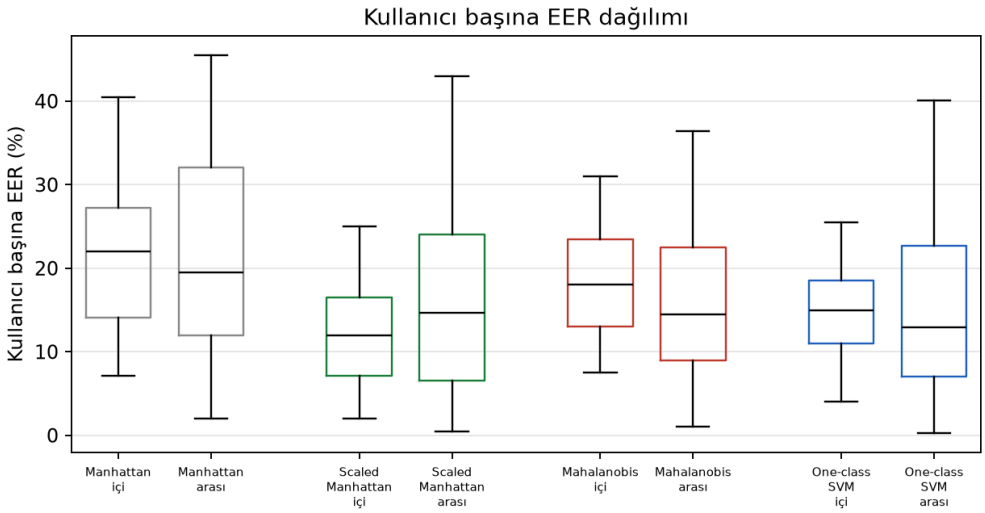
6.5 Gerçekçi eşik: eşiği yalnızca eğitim döneminden seçersek?

EER eşiği test verisine bakarak seçer, gerçek sistemde bu olmaz. Bu yüzden eşiği yalnızca eğitim oturumlarından seçtik: eğitim oturumlarını tek tek dışarıda bırakıp (oturum-bırakmalı çapraz doğrulama) kullanıcının kendi skorlarının %90'lık niceliğini eşik aldık; yani gerçek kullanıcıların %90'ı geçsin istedik. Sonra oturum 5–8'de test ettik:

Dedektör	Hedef: gerçek kullanıcıların %90'ı geçsin → gerçekleşen FNMR	Gerçekleşen FMR (yabancı kabul)
Manhattan	%6,4	%55,4
Scaled Manhattan	%6,4	%37,5
Mahalanobis	%6,0	%42,5
One-class SVM	%9,0	%26,6

Yorum: Gerçek kullanıcıların yaklaşık %6–9'u reddediliyor (hedeften iyi, eşik biraz gevşek), ama yabancıların %27–55'i kabul ediliyor. Yani tek başına bir güvenlik katmanı olarak zayıf; demoda bunu dürüstçe söylemeliyiz ("parola yanlışsa zaten girilemez, ritim ek bir engel"). Bu aynı zamanda demo için eşik seçme yöntemimiz olacak.

6.6 Kullanıcı farklılığı



Şekil 4. Kullanıcı başına EER dağılımı (kutu: çeyrekler, çizgi: medyan). Bazı kullanıcılar neredeyse %0, bazıları %40'ın üzerinde.

Dağılım çok geniş: bazı kullanıcıların ritmi çok ayırt edici, bazılarınınki neredeyse yok. Ortalama bunu saklar, o yüzden raporda dağılımı ve en kötü kullanıcıların hata analizini vereceğiz (hocanın "hataları analiz et" şartı).

7. Demo: localhost giriş ekranı

Fikir yapılabilir mi? Evet. Önerdiğin akış, projenin kendi deney tasarımının küçük bir kopyası: oturum 1 = eğitim, sonraki gün oturum 2 = hem test hem gösterilebilir bozulma.

7.1 Akış

1. **Kayıt (Enroll) ekranı:** Kullanıcı seçilir (Bahadır / Ömer Efe), "Oturum 1" seçilir, parola 50 kez yazılır. Tarayıcı her giriş için `keydown` ve `keyup` zamanlarını (`performance.now()`) kaydeder; H, DD, UD özellikleri hesaplanır ve kaydedilir.
2. Bir gün sonra "Oturum 2" aynı şekilde yapılır. Böylece her kullanıcıda CMU'daki yapıya benzer, gün bazında ayrılmış veri oluşur.
3. **Eğitim:** Kullanıcının kayıt verisiyle ona özel dedektör (One-class SVM, istersek karşılaştırma için Scaled Manhattan) eğitilir; saniyeler sürer. Eşik, bölüm 6.5'teki yöntemle (oturum-bırakmalı) yalnızca kayıt verisinden seçilir.
4. **Giriş (Login) ekranı:** Kullanıcı adı seçilir, parola yazılır. Sistem parolayı *ve* yazış ritmini kontrol eder, skoru ve kararı (kabul/ret) gösterir.
5. **Saldırı gösterimi:** Parola ekranda yazılıdır; başkası (ör. Ömer, Bahadır'ın hesabına) aynı parolayı yazar ve ritmi uymadığı için reddedilir.

7.2 Parola: aynı olsun mu, farklı mı?

> **Önerim: iki kullanıcı için de aynı parola, `bahadir23.`**
>
> - **Neden aynı?** Farklı parolada (`bahadir23.` ve `omerefe123.`) birinin hesabına başka parolayla girmek zaten parola yanlış olduğu için reddedilir; ritim hiç test edilmez. Aynı parolada tek ayırt edici şey yazış ritmidir, projenin iddiası tam olarak bunu gösterir. Üstelik CMU verisi de "herkes aynı parolayı yazar" tasarımıyla toplanmış.
> - **Güzel bir tesadüf:** `bahadir23.` 10 karakter + Enter = 11 tuş, yani CMU'nun `.tie5Roanl` parolasıyla aynı yapıda: 11 H + 10 DD + 10 UD = **31 özellik**. Bu yüzden özellik çıkarım kodunu ve CMU'da seçtiğimiz hiperparametreleri (ν=0.05, γ=0.03) hiç değiştirmeden kullanabiliriz. `omerefe123.` 11 karakter + Enter = 12 tuş = 34 özellik olurdu; teknik olarak mümkün ama ekstra uyarlama gerekir.
> - İstersen `omerefe123.` ile ek bir "farklı parola" senaryosu da gösterebiliriz, ama ana demo ortak parolayla olsun.

7.3 "Modeli önceden eğitelim" ne demek?

CMU'da eğitilen model demodaki kullanıcılara **uygulanamaz**: dedektörler kişiye özel; Bahadır'ın ritmi CMU'da yok. Önceden hazır olan şey **hattın kendisi**: özellik çıkarımı, doğrulanmış dedektör kodu, CMU'da seçilmiş ν ve γ, eşik seçme yöntemi. Kişisel modeller kayıt bittiğinde saniyeler içinde eğitilir. (İkinci geliştirmedeki paylaşımlı gömü ağı ise CMU'da önceden eğitilebilir, bölüm 8.)

7.4 Sınırlar ve riskler

- **Zamanlama farkı:** CMU verisi harici bir saatle toplandı; tarayıcı olay zamanları (klavye, işletim sistemi, tarayıcı) farklı gecikme taşır. Demo verisi bu yüzden CMU'yla karıştırılmaz ve doğruluk

iddiası için kullanılmaz; demo sadece hattın çalıştığını gösterir.

- **Veri azlığı:** İki kullanıcı × birkaç oturum çok az; demodaki ret/kabul oranı istatistiksel kanıt değil. Raporda bunu açıkça yazacağız; ciddi sayılar CMU deneylerinden gelir.
- **Hoca onayı:** Proje metni "kendi biyometrik verini toplamak için hocanın önceden onayı gerekir" diyor. Demo verisi yalnızca ikimizden toplanacak olsa da, veriyi toplamadan önce hocaya kısa bir e-posta ile sormalıyız. Onay gelene kadar demoyu CMU'dan seçilmiş bir kişinin verisiyle simüle edecek şekilde de hazırlayabiliriz.
- **Başarı garantisi yok:** Biz de dahil, bazı insanların ritmi tutarsızdır (Şekil 4). Demoda Bahadır reddedilirse bunu sistemin sınırı olarak anlatırız; ama önceden denemek şart.

7.5 Teknik taslak

Tarayıcıda küçük bir JavaScript (`keydown/keyup` yakalama), arkada Python **Flask** sunucusu (zaten kurulu), veriler SQLite veya JSON dosyasında. Dedektör kodu doğrudan `keystroke/detectors.py`'den yüklenir; yani demo ile deneyler aynı kodu paylaşır.

---

8. İkinci geliştirme: paylaşımlı gömü ağı (isteğe bağlı)

Önceliğimiz SVM; bunu hocaya "ek geliştirme" olarak sunacağız. Fikir: tek bir küçük sinir ağını (Siamese/üçlü kayıpla) birçok kişinin verisiyle eğitip her girişi bir "ritim vektörüne" çevirmek; sonra kullanıcı kaydı = birkaç vektörün ortalaması, doğrulama = vektör mesafesi.

> **Hocanın kuralı:** "Kullanıcılar arasında paylaşılan öğrenen model, değerlendirmede kullanılmayan kullanıcılarla eğitilmelidir." Çözüm: kişileri 5 gruba bölüp her grubu, *o grubu hiç görmemiş* ağla değerlendirmek (çapraz katlama). Böylece bütün 51 kişiyi dürüstçe değerlendirebiliriz. Bu daha fazla iş ve süre alacağı için SVM sonuçları oturduktan sonra yapacağız; yetişmezse "gelecek çalışma" olarak yazarız.

9. Hocanın kurallarına karşı kontrol listesi

Kural	Durum
Kendi uyguladığımız en az bir aşama	Tamam: 4 dedektör, metrikler ve deneyler kendi kodumuz
Açık kaynak parçaların beyanı	scikit-learn (OneClassSVM çekirdeği), numpy, pandas, matplotlib, Flask beyan edilecek
Eğitim/test kişi ve zaman ayrımı	Tamam: zamana göre bölme + ayrık ayar grubu
Eşiğin hangi veride seçildiğini yazmak	Tamam: EER = test skorlarından (makale gibi), gerçekçi eşik = yalnızca eğitim dönemi
Desteklenen düzeyde hata oranı	FMR en fazla %0,1'e kadar
Belirsizlik (denek bootstrap)	Tamam; zamanla bozulma eğrisi için eksik, eklenecek
Hata analizi, makaleyle karşılaştırma	Karşılaştırma tamam; en kötü kullanıcıların analizi eksik
En az 10 kaynak, hepsi gerçek ve doğrulanmış	Henüz yok. Yapay zekâ kaynak uydurabilir; her kaynağı arayıp doğrulayacağız ve yalnızca bulduklarımızı yazacağız.
AI Use Declaration (tasarım dokümanı ve raporun sonunda)	Yazılacak: hangi araç, hangi iş, nasıl denetlendi
Kendi veri toplama onayı	Hocaya sorulacak (bölüm 7.4)
Kod ZIP: README, requirements, sonuçları yeniden üreten betik	`requirements.txt` ve `run_all.py` hazır; README sonra

10. Yol haritası

Tarih	Ne	Durum
Hemen	Hocaya demo verisi için onay e-postası	Bekliyor
30 Ekim	**Tasarım dokümanı** (2–3 sayfa PDF): literatür, şema, veri/lisans, bölme kuralı, metrikler, araçlar, AI beyanı	Sıradaki iş; 10 kaynak taraması ve doğrulama
9 Kasım haftası	Veri ve ön işleme sunumu (5 dk): veri istatistikleri, kalite sorunları, reddedilen örnek	İçerik büyük oranda hazır (bölüm 3)
27 Kasım	İlk sonuçlar: skor dağılımları ve DET	Şimdiden var; skor dağılım grafiği eklenecek
Kasım	Demo uygulaması (kayıt, eğitim, giriş), ikinci geliştirme	Başlanmadı
7 Aralık haftası	Final sunum ve demo (10 dk + 5 dk soru)	—
11 Aralık	Rapor (10–15 sayfa) ve kod ZIP	—

Sonraki adımım için öneri: (1) Tasarım dokümanını birlikte yazmaya başlamak; önce kaynak taraması, sonra 2–3 sayfalık PDF. (2) Paralelde demo uygulamasının iskeletini kurmak.

Sözlük

Terim	Anlamı
Genuine / gerçek kullanıcı	Kendi hesabına giren kişi
Impostor / yabancı	Başkasının hesabına girmeye çalışan kişi
Anomali skoru	Girişin kullanıcının normalinden ne kadar uzak olduğu
Oturum	Bir günde yapılan 50 giriş
Bootstrap	Veriyi yerine koyarak yeniden örnekleyip belirsizliği ölçme yöntemi
Hiperparametre	Öğrenmeden önce elle seçilen ayar (SVM'de ν ve γ)
Sızıntı	Test bilgisinin yanlışlıkla eğitime karışması, sonucu olduğundan iyi gösterir

Kaynaklar (bu belgede kullanılanlar): K. S. Killourhy ve R. A. Maxion, "Comparing Anomaly-Detection Algorithms for Keystroke Dynamics", DSN 2009 (proje klasöründeki PDF'ten okundu); hocanın "Biometric Recognition Project" ve "Project Topics" belgeleri (30 Eylül 2026). Bu belgedeki bütün sayılar `results/results.json` dosyasından otomatik üretilmiştir. Belgenin hazırlanmasında Claude (Anthropic) kod yazma ve metin taslağı için kullanıldı; sonuçlar makale değerleriyle yeniden üretilerek denetlendi.